COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S1
DATE PERFORMED:	10/15/26	**DATE SUBMITTED:**	10/20/26
MEMBER/S:	Egar, Jarel Dave Medina, Jomz Ryle E.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 13.1 **Parallel Algorithms and Multithreading**

1. PROCEDURE OUTPUT

ILO A & ILO B Output/s:

Screenshot

Code Screenshot:

```cpp
#include <iostream>
#include <thread>
#include <string>

void print(int n, const std::string &str) {
    std::cout << "Printing integer: " << n << std::endl;
    std::cout << "Printing string: " << str << std::endl;
}

int main() {
    std::thread t1(print, 10, "T.I.P.");
    t1.join();
    return 0;
}
```

Code Output (separately run on different computers):

Egar:

```
Printing integer: 10
Printing string: T.I.P.
```

Medina:

```
Printing integer: 10
Printing string: T.I.P.
```

Analysis	Egar:	This demonstrated how a Simple one threaded function by creating a thread that prints integer and TIP. This code made use of th.join but what does th.join do? Sir Jim taught us that this code will only run after the worker function is finished with its job then it will run the other code. If you were to remove th.join it will give you an error because the program will terminate instantly.
	Medina:	The screenshot above shows the implementation of a basic simple one-threaded program. It creates a worker thread, "**t1**", to execute the **print()** function. The program additionally uses the **t1.join()** function to make the main thread wait for thread **t1** to finish before returning 0 as taught in the lecture. The terminal output remains very similar across multiple executions because the integer and string print statements are executed sequentially inside the single t1 thread.

Table 13-1. Simple One-Threaded Example

Screenshot

Code Screenshot:

```cpp
#include <iostream>
#include <thread>
#include <string>
#include <vector>

void print(int n, const std::string &str) {
    std::string msg = std::to_string(n) + " : " + str + "\n";
    std::cout << msg;
}

int main() {
    std::vector<std::string> s = {
        "T.I.P.",
        "Competent",
        "Computer",
        "Engineers"
    };

    std::vector<std::thread> threads;

    for (int i = 0; i < s.size(); i++) {
        threads.push_back(std::thread(print, i, s[i]));
    }

    for (auto &th : threads) {
        th.join();
    }

    return 0;
}
```

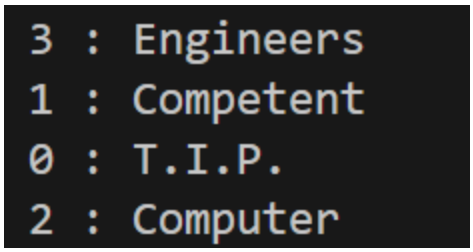
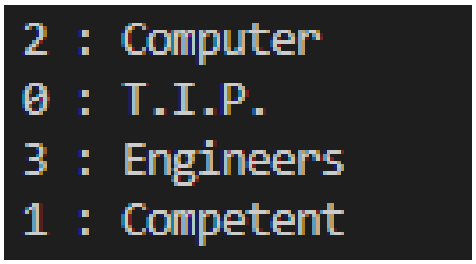
<table>
<tr><td></td><td colspan="2"><table>
<tr><td colspan="2">Code Output (separately run on different computers):</td></tr>
<tr><td>Egar:</td><td> 
</td></tr>
<tr><td>Medina:</td><td> 
</td></tr>
</table></td></tr>
<tr><td>Analysis</td><td>Egar:</td><td>On the other hand, this part showed a multithread function. Basically this code used the th.join as well and this actually provided me with a much more clearer example. This activity has 4 workers which is computer, TIP, Engineers, and components, the th.join will wait for 1 worker to finish then it will run the other. After running it multiple times I noticed that each output is different from the original as each time I ran the code other worker functions finished before the previous first output.</td></tr>
<tr><td></td><td>Medina:</td><td>The code above now shows that of the multithreaded example as opposed to the simple one-threaded one. Four threads are now used and secured with th.join() where all of these have to complete before main returns 0. The terminal varies between Egar and I’s outputs as multithreading executes at the same time and it is no longer sequential, outputting whatever finishes first as taught in the lecture and also explained in the lab.</td></tr>
</table>

Table 13-2. Multithreaded Example.

2. SUPPLEMENTARY OUTPUT

Part A:	
1. Write a definition of multithreading and its advantages/disadvantages.	
Egar:	Multithreading is a programming method where multiple callable functions are in 1 thread. Threads are a collection of processes that share system resources. A single thread executes code one after the other, while multithreading lets multiple threads run inside one process so the computer can handle several jobs at once. However, multithreading can cause bugs and slow down your computer if threads try to use the same data at the same time or spend too much time switching between tasks.
Medina:	Multithreading is basically an ability that allows a single program to use multiple 'threads' which are just paths of execution that share the said program's memory and resources. Advantages include faster processing with the help of multithreading across different CPU cores and possible crashes/errors in the case that the resources are not managed well.
2. Rationalize the use of multithreading by providing at least 3 use-cases.	
Egar:	First example of this is a video and audio editor software; they use multiple threads in order to render all of the effects that are in use by the user. Second is OS or Operating Systems, this is a given because an OS processes a lot of things in the background that help run the PC more smoothly. Lastly, video games, video games use multiple threads so the game can process graphics, sound, and player movement all at the same time without lagging.
Medina:	A use-case for multithreading is with the servers that we use everyday from our games to the websites that we visit where a computer does a complex task for one user while showing the result for another. Moreover, another use case would be with graphics processing, like with rendering an edited video and how portions of it could be processed in parallel across multiple CPU cores. Finally, a third use case is for applications themselves where a game could be calculating background money earned by the user while manipulating the interface based on input.
3. Differentiate between parallelism and concurrency.	
Egar:	Parallelism is when the system is doing multiple tasks at the same time, and it uses 2 different CPU cores however, concurrency is all about handling multiple tasks and switching between 1 core back and forth,
Medina:	Parallelism is when multiple tasks go at the same time via multiple CPU cores whereas concurrency is kind of like switching between multiple tasks, using a single CPU core.

Part B:

Code Screenshot:

13p1.cpp > ...

```cpp
#include <iostream>
#include <thread>

int total = 0;

void add(int number) {
    total = total + number;
}

int main() {
    std::thread T1(add, 10);
    std::thread T2(add, 20);
    std::thread T3(add, 30);

    std::cout << "Before joining: " << total << '\n';

    T1.join();
    std::cout << "After T1: " << total << '\n';

    T2.join();
    std::cout << "After T2: " << total << '\n';

    T3.join();
    std::cout << "After T3: " << total << '\n';

    return 0;
}
```

Output:

```
Before joining: 0
After T1: 60
After T2: 60
After T3: 60
```

Analysis

Egar: This code shows how multithreading works by creating a worker functions that add 10, 20, and 30 together. In the code, there is a function called T1.join, this prints the total before the thread is fully joined.

Medina: Upon running multiple tests of the code written above, 60 seemed to consistently display after each of the **join()** calls. At first, I was confused by this as I was expecting it to show 10, 30, then 60, but upon further understanding, I realized that **join()** only waits for the completion, not changing the displayed value with the threads running concurrently within the background. Thus, all the threads finish updating the total, '60' even before "After T1: …" is displayed.

Part C:		

Code Screenshot:	13p12.cpp > ...
	`#include <iostream>`
	`#include <thread>`
	
	`// Straight copied and slightly modified from HOA 6.1 (Linear Search Algo)`
	`void linearSearchArr(int data[], int arrSize, int findNum, int *result) {`
	`  int j = 0; // comps`
	`  int i = 0; // counts`
	
	`  while (i < arrSize) {`
	`  if (findNum == data[i]) {`
	`  j++;`
	`  }`
	`  i++;`
	`  }`
	
	`  *result = j;`
	`}`
	
	`int main() {`
	`  int data[] = {6, 7, 4, 2, 0, 5, 8, 1, 5, 4, 6, 5};`
	`  int findNum = 5;`
	
	`  int result1 = 0;`
	`  int result2 = 0;`
	`  int result3 = 0;`
	
	`  std::thread T1(linearSearchArr, &data[0], 4, findNum, &result1);`
	`  std::thread T2(linearSearchArr, &data[4], 4, findNum, &result2);`
	`  std::thread T3(linearSearchArr, &data[8], 4, findNum, &result3);`
	
	`  T1.join();`
	`  T2.join();`
	`  T3.join();`
	
	`  std::cout << "Thread 1 matches: " << result1 << '\n';`
	`  std::cout << "Thread 2 matches: " << result2 << '\n';`
	`  std::cout << "Thread 3 matches: " << result3 << '\n';`
	
	`  int total = result1 + result2 + result3;`
	
	`  std::cout << "The element '" << findNum << "' is found " << total << " time/s.\n";`
	
	`  return 0;`
	`}`	

Output:	`Thread 1 matches: 0`
	`Thread 2 matches: 1`
	`Thread 3 matches: 2`
	`The element '5' is found 3 time/s.`	

Analysis	Egar:	We used a linear search for part C. by splitting the array, which has 12 elements,

		into 3 small parts of 4 elements, we then created 3 separate threads that will be searching its designated section at the same time. Each thread counts how many times the number 5 appears in its section, which then adds up to a total of 3 occurrences across the whole array. Because all 12 elements still need to be searched, the overall time complexity remains O(n), but the process completes faster due to parallel execution.
	Medina:	For part C, we implemented code in a way that uses linear search across multiple threads after splitting an array into three portions. Each of the threads check for four elements each and eventually add up together to find a total of **3 occurrences** for the value, '**5**'. Additionally, all of the 12 elements are checked so the total time complexity should still be about O(n).

Conclusion/s:

Egar:

In conclusion, this HOA showed me how multithreading works, based on the activities. In Part A, I learned the basics of multithreading, its advantages, disadvantages, and real-world examples like video games, operating systems, and media editors, as well as how concurrency switches between tasks while parallelism runs tasks at the exact same time on different CPU cores. Part B showed me how multiple threads can run in the background to modify shared data and how join() waits for threads to finish so the main program doesn't terminate early or lose track of updates. Finally, Part C demonstrated how we can apply multithreading to speed up algorithms like linear search by dividing a 12-element array among 3 threads, making the search run in parallel while keeping the same overall logic.

Medina:

In this activity, we learned a lot about concurrency, parallelism, and how multithreading is applied in a program. The **join()** function makes sure that a thread finishes before the main program continues. Additionally, the exercises allowed me to envision that shared variables require proper syncing despite the output appearing correct. Moreover, the linear search implemented in part C of the supplementary had us apply multithreading in three sections of a single array, storing the results independently, and correctly finding the amount of occurrences of the target value. Overall, I believe I did well in the activity as it showed me the basics of multithreading and how to actually use it practically in dividing the work among threads.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **None**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

COMPUTER ENGINEERING

QUEZON CITY

Purpose of Use: **N/A**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **N/A**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.